

任务四：班级成绩分布（高级直方图与箱线图）

生成模拟数据

第一部分：代码逐段白话详解

1. 魔法前置：锁死“时光机”

```
import numpy as npimport matplotlib.pyplot as plt# 中文显示设置（略）
np.random.seed(42)
```

详解：`np.random.seed(42)`这行代码非常神奇。因为计算机生成的随机数其实是“伪随机”，只要我们设定了一个固定的“种子（seed）”（这里的 42 只是个代号，写 1、100、888 都可以），那么每次运行这段代码，它生成的 40 个分数将永远一模一样。

为什么这么做？方便你和同桌对照图表。如果大家图长得不一样，就没法讨论代码对不对了。

2. 核心造数：捏出成绩的“灵魂”

```
scores = np.random.normal(75, 10, 40)
```

详解：`np.random.normal()` 是用来生成“正态分布”数据的。什么是正态分布？就是真实考试中最常见的现象：考中等分数的人最多，特别高和特别低的人很少。

里面三个参数的含义：

75：均值（平均分）。把大部队的分数定在 75 分左右。

10：标准差（离散程度）。数字越大，成绩两极分化越严重；数字越小，大家成绩越抱团。

40：数量。我们需要生成 40 个同学的成绩。

3. 边界修剪：让成绩符合现实法则

```
scores = np.random.clip(scores, 0, 100)# 这里原代码写的是 np.clip，效果一样
```

详解：正态分布在数学上是没有边界的！它完全有可能给你随机出一个 105 分，或者 -2 分。真实考试中显然不可能。`np.clip(数据, 最小值, 最大值)` 的作用就是拿着一把剪刀：凡是低于 0 分的，强行拔高到 0 分；凡是超过 100 分的，强行削平到 100 分。

4. 格式整形：把带小数的成绩变整数

```
scores = np.round(scores).astype(int)
```

详解：经过上面几步，生成的成绩其实是带长长小数的（比如 75.34125 分）。

`np.round(scores)`：先进行四舍五入（变成 75.0）。

`astype(int)`：彻底脱掉小数点的外衣，把数据类型强制转换成整数（变成 75）。

5. 验货：打印出来看一眼

```
codePython
```

```
print("前 10 个成绩：", scores[:10])
```

```
print("最高分：", scores.max())
```

```
print("最低分：", scores.min())
```

```
print("平均分：", scores.mean())# 原代码这里漏了括号！
```

详解：在画图之前，先用 `print` 和 NumPy 自带的数学统计函数（`.max()`、`.min()`、`.mean()`）把数据打印在控制台看一眼，确保我们的数据生成是成功的。

这段代码虽然短，但涉及了 NumPy 的高级用法，非常容易出错：

易错点 1：括号不匹配（SyntaxError）

错误做法：写成 `print("平均分：", scores.mean()`

后果：这是所有新手最容易犯的低级错误！Python 报 `SyntaxError: unexpected EOF while parsing`（语法错误：解析时遇到意外的结尾）。

牢记：只要写了左括号（`,`），就一定要有对应的右括号（`)`）。尤其是在函数套函数的时候（比如 `scores.mean()` 自身带了一对括号，外面 `print` 还要有一对大括号）。

易错点 2：搞错 `normal` 函数的参数顺序

错误做法：记错顺序，写成了 `np.random.normal(40, 75, 10)`。

后果：代码不会报错，但你生成了 10 个学生的数据，他们的平均分是 40，标准差高达 75！这会导致大量数据变成负数然后被强制变成 0 分，后面的图表完全失去意义。

牢记：顺序永远是（平均值，标准差，数据个数）。

易错点 3：觉得 `clip` 没用把它删了

错误做法：心想“平均分才 75，怎么可能超 100”，于是删掉 `np.clip` 这行。

后果：正态分布的尾巴是很长的。当你以后把平均分改成 90，标准差改成 15 时，必然会生成出 110 分的成绩。如果不做修剪，后面的直方图 X 轴会跑到 100 以外，被老师一眼看出数据造假。

易错点 4：忘记写 `.astype(int)`

错误做法：只写了 `scores = np.round(scores)`。

后果：数据依然是浮点数格式（比如 75.0, 82.0）。虽然在画直方图时不会报错，但如果你打印出来看，会发现非常不美观，且在后续做一些更精细的文本标注时，容易产生格式干扰。

绘制成绩分布直方图

第一部分：代码逐段详解

1. 常规操作：铺画布、写标题、画网格

```
plt.figure(figsize=(10, 5)) # 创建一张宽 10、高 5 的大画布
```

```
# 添加标题和坐标轴标签
```

```
plt.title('班级数学成绩分布直方图 (n=40)', fontsize=15)
```

```
plt.xlabel('分数段', fontsize=12)
```

```
plt.ylabel('学生人数（频数）', fontsize=12)
```

```
# 拉一排水平虚线网格，方便肉眼对齐柱子高度
```

```
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

详解：这些都是常规的图表“精装修”，让图表看起来更专业易读。

2. 核心神技：获取 `plt.hist` 的隐藏返回值

```
# 这行代码极其关键！它既在画图，又在输出数据
```

```
n, bins, patches = plt.hist(scores, bins=8, color='skyblue', edgecolor='black', alpha=0.7)
```

详解：

以前我们只写 `plt.hist(...)` 让它把图画出来就行了。

但这次，我们在它前面加了 `n, bins, patches =`。意思是：`plt.hist` 在干活的时候，还会偷偷汇报三个结果，我们用三个变量把它们接住！

n(频数列表)：存放着每根柱子的高度（人数）。比如 `[2, 5, 10...]`。

bins(边界列表)：存放着每个区间切分的边缘刻度。比如 `[40, 50, 60...]`。

patches：存放着画出来的矩形图形本身（在这个打标签的任务里，我们用不到它，但按语法必须写上接住）。

注意一个小瑕疵：你在代码开头已经写过一次 `plt.hist(...)` 了，这里为了获取返回值又写了一次。实际上，画两次是一样的，它们会重叠在一起，视觉上看不出来，但正确的做法是只保留这一行带返回值的就足够了。

3. 动态打标签：烧脑的数学计算

```
for i in range(len(n)):
```

```
    # 这是全场最复杂的数学逻辑！
```

```
    plt.text(bins[i] + (bins[1] - bins[0])/2, n[i] + 0.5, int(n[i]), ha='center', fontsize=9)
```

详解：`len(n)` 就是柱子的总数。循环依次给每根柱子戴上“数字帽子”。我们要告诉 `plt.text(X 坐标, Y 坐标, 要写的文字)`。

Y 坐标（高度）：`n[i] + 0.5`。`n[i]` 是当前柱子的高度，加 `0.5` 是为了让数字悬空一点，别压着柱子顶端的线。

要写的文字：`int(n[i])`。把人数转成纯整数写上去。`ha='center'` 确保文字居中对齐。

X 坐标（水平位置）【最难点】：`bins[i] + (bins[1] - bins[0])/2`

我们要在柱子的水平正中心写字。

`bins[i]` 是这根柱子的左边界位置（比如 60 分）。

`(bins[1] - bins[0])` 是用第二个刻度减第一个刻度，算出了一个区间的总宽度（比如 10 分）。

总宽度除以 2，就是半个区间的宽度（5 分）。

左边界位置 + 半个区间的宽度 = 柱子的正中心！（`60 + 5 = 65` 的位置）。

这段代码非常高级，稍微改错一点就满盘皆输：

易错点 1：把直方图的打标签公式，和柱状图的搞混！

错误做法：在直方图里写 `plt.text(i, n[i] + 0.5, ...)`。

后果：数字会全部错位，跑到图表的最左边去！

牢记原因：在普通的柱状图（`bar`）中，柱子的位置就是 `0, 1, 2` 索引位置，所以 `X 坐标` 直接写 `i` 就行。但直方图的 `X 轴` 是真实的数值刻度（比如 60 分, 75 分），不是索引 `0, 1, 2`！

所以必须用上面那一长串数学公式去计算真实的中心点 `X 坐标`。

易错点 2：接收变量的顺序写错

错误做法：写成 `bins, n, patches = plt.hist(...)`

后果：下面循环计算时，把 `n` 当成了边界，把 `bins` 当成了人数，直接报一长串数学运算错误。

牢记：`plt.hist` 返回值的顺序是雷打不动的：频数(`n`) -> 边界(`bins`) -> 图形(`patches`)。必须严格按这个顺序接收。

易错点 3：打出来的文字带小数

错误做法：最后面文字参数直接写 `n[i]`。

后果：图表柱子头上会出现 10.0 这种带小数的人数。虽然数值对，但“半个人”非常反人类。
牢记：`plt.hist` 吐出来的频数 `n` 默认带有浮点数属性。打标签时务必套上 `int()`，比如 `int(n[i])`，强制转成干净的整数。

易错点 4：重复画图导致的模糊

不良习惯：如你在原代码中所写，不带返回值画了一次 `plt.hist`，为了接返回值又画了一次 `plt.hist`。如果这两次画的 `color` 或者 `alpha` 参数稍微不一样，图表就会发生灵异的变色或边缘模糊，因为你在同一个地方盖了两层楼。

最佳实践：只画一次，边画边接返回值。把原代码前面单独的 `plt.hist(...)` 删掉即可。

绘制成绩分布箱线图

第一部分：代码逐段详解

1. 铺垫工作与神仙字体设置

```
import numpy as np
import matplotlib.pyplot as plt
```

这行设置中文字体的代码非常“豪华”

```
plt.rcParams['font.sans-serif'] = ['PingFang SC', 'Heiti TC', 'SimHei', 'Arial Unicode MS']
```

```
plt.rcParams['axes.unicode_minus'] = False
```

详解：之前我们只写 `['SimHei']`。但这里写了一个长长的列表。为什么？

因为不同电脑系统自带的字体不一样！`PingFang SC` 和 `Heiti TC` 是苹果 `Mac` 电脑专属，`SimHei` 是 `Windows` 专属。

`Python` 会像个乖宝宝一样，从左到右依次去找。如果在 `Mac` 上，它找到第一个就用了；如果在 `Win` 上，前两个找不到，它就会顺延找到第三个 `SimHei`。这是一种兼容性极强的防报错写法。

2. 数据与画布准备

```
plt.figure(figsize=(6, 8)) # 宽 6，高 8。箱线图通常偏长条形，所以高比宽大。
```

手动给出一组 10 个人的成绩数据

```
scores = [80, 74, 81, 90, 73, 73, 91, 83, 70, 80]
```

3. 核心绘图：高度定制的箱线图

```
codePython
```

这行代码长得可怕，但拆开看其实很简单！

```
plt.boxplot(scores, patch_artist=True, boxprops={'facecolor': 'moccasin'},
medianprops={'color': 'red', 'linewidth': 2})
```

详解：`plt.boxplot()` 是画箱线图的专属函数。后面带了一长串修饰参数：

`patch_artist=True`：非常关键的一句魔法咒语！默认画出来的箱线图，箱子内部是透明空心的。只有念了这句咒语，系统才允许你给箱子“涂色”。

`boxprops={'facecolor':'moccasin'}`: 给**箱子本身 (box) 的属性 (props) **定规矩。传入一个字典, 要求它的“脸的颜色”(内部填充色)是 `moccasin` (一种好看的浅鹿皮色/淡黄色)。
`medianprops={'color':'red', 'linewidth': 2}`: 给**中位线 (median) 的属性 (props) **定规矩。要求这条横穿箱子中心的线, 颜色为红, 线条宽度为 2 (加粗, 更醒目)。

4. 图表精装修: 标签与网格

```
plt.title('班级数学成绩箱线图',fontsize=15)
```

```
plt.ylabel('分数', fontsize=12)
```

```
# 强制替换 X 轴的默认名字
```

```
plt.xticks([1], ['全班成绩'])
```

```
# 拉水平虚线网格
```

```
plt.grid(axis='y', linestyle='--', alpha=0.6)
```

详解: `plt.xticks([1], ['全班成绩'])` 这一步很有意思。

因为我们只给了一组数据 `scores`, 所以画出来只有一个箱子。系统会默认在 X 轴的刻度 1 的位置画, 并在下面标一个傻傻的数字 1。

这句话的意思是: “去刻度 1 的位置, 把那个数字删了, 给我换成文字『全班成绩』”。

这段代码中字典的层层嵌套最容易引发语法灾难:

易错点 1: 想要彩色箱子, 却忘了念咒语 `patch_artist=True`

错误做法: 直接写 `plt.boxplot(scores, boxprops={'facecolor':'red'})`。

后果: 代码直接报错崩溃。因为系统不认识这画笔, 它默认你只能用线条款式, 不能用水彩笔填涂面。

牢记: 想填色, 先加 `patch_artist=True`。

易错点 2: 字典的括号 `{}, [], ()` 大乱炖

错误做法: `boxprops=(facecolor='moccasin')` 或者 `boxprops={'facecolor','moccasin'}`

后果: 经典的 `SyntaxError` 语法错误。

牢记: 属性配置一定要用字典 (大括号 `{}`)! 而且字典里面的规矩是 '键': '值' (必须有冒号)。比如 `'color': 'red'`。

易错点: 把 `xticks` 和 `xlabel` 搞混

错误做法: 想把横轴标成“全班成绩”, 写成了 `plt.xlabel('全班成绩')`。

后果: 图表底部的中间会多出“全班成绩”四个大字, 但那个孤零零、傻乎乎的数字 1 依然坚挺地站在箱子正下方。

牢记区分:

`xlabel` 是给整条横轴起个大标题。

`xticks` 是去修改横轴上某一个具体小刻度对应的标签。对于只有一组数据的箱线图, 通常我们需要用 `xticks` 把那个刻度 1 给盖掉。